# Perulangan 2

**Mata Kuliah Dasar Pemrograman**
**Jurusan Teknologi Informasi**
**2026**

# Tujuan

Di akhir pertemuan, mahasiswa diharapkan mampu:

- Memahami konsep perulangan bersarang pada algoritma
- Mahasiswa mampu menggambarkan flowchart perulangan bersarang
- Mengenal sintaks perulangan bersarang
- Mampu menerapkan perulangan bersarang dalam studi kasus

# *Outline*

- Perulangan Bersarang
- Studi kasus

# *Preface*

- Dalam konsep dasar perulangan, logika perulangan digunakan untuk melakukan beberapa **proses/*statement*** program secara **berulang-ulang**, dengan suatu **pola tertentu**.

- **Proses/*statement*** akan terus dilakukan secara **berulang-ulang**, selama **kondisi perulangan** bernilai **benar/true**. Dan sebaliknya, perulangan akan **berhenti** dan proses/*statement* tidak akan dieksekusi lagi ketika **kondisi perulangan** bernilai **salah/false**.

- **Kondisi perulangan** *(syarat perulangan)* diperlukan untuk menentukan apakah suatu perulangan masih akan berlangsung lagi atau harus berhenti.

# Definisi

- Perulangan bersarang (***nested loop***) adalah
  - ✓ struktur perulangan yang berada di dalam perulangan lainnya, ***atau***
  - ✓ suatu perulangan yang memiliki perulangan lagi di dalamnya.

- *Loop* terluar dikenal dengan istilah ***outer loop,*** sedangkan *loop* yang ada di dalamnya disebut ***inner loop***.

- *Nested loop* bisa lebih dari 2 tingkat/level *(minimal 2 tingkat/level)*

# *Pseudocode Nested Loop*

- *Nested loop* bisa memiliki lebih dari 2 tingkat.
- Secara umum gambaran *nested loop* seperti berikut:

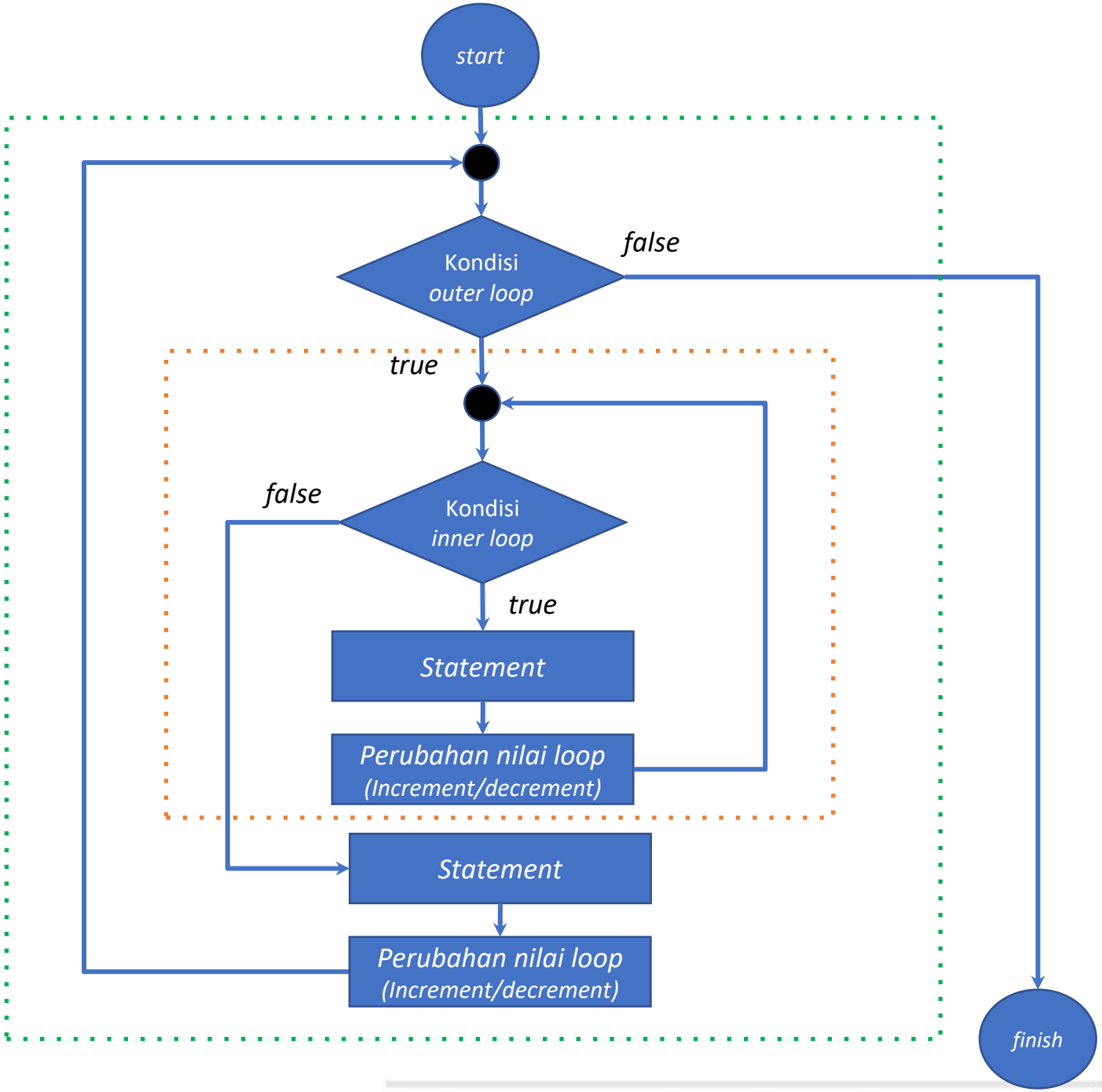
```
1  loop-level-1 {
2      loop-level-2 {
3          .....
4          loop-level-n {
5              // statement
6          }
7          .....
8      }
9  }
```

# *Flowchart Nested Loop*

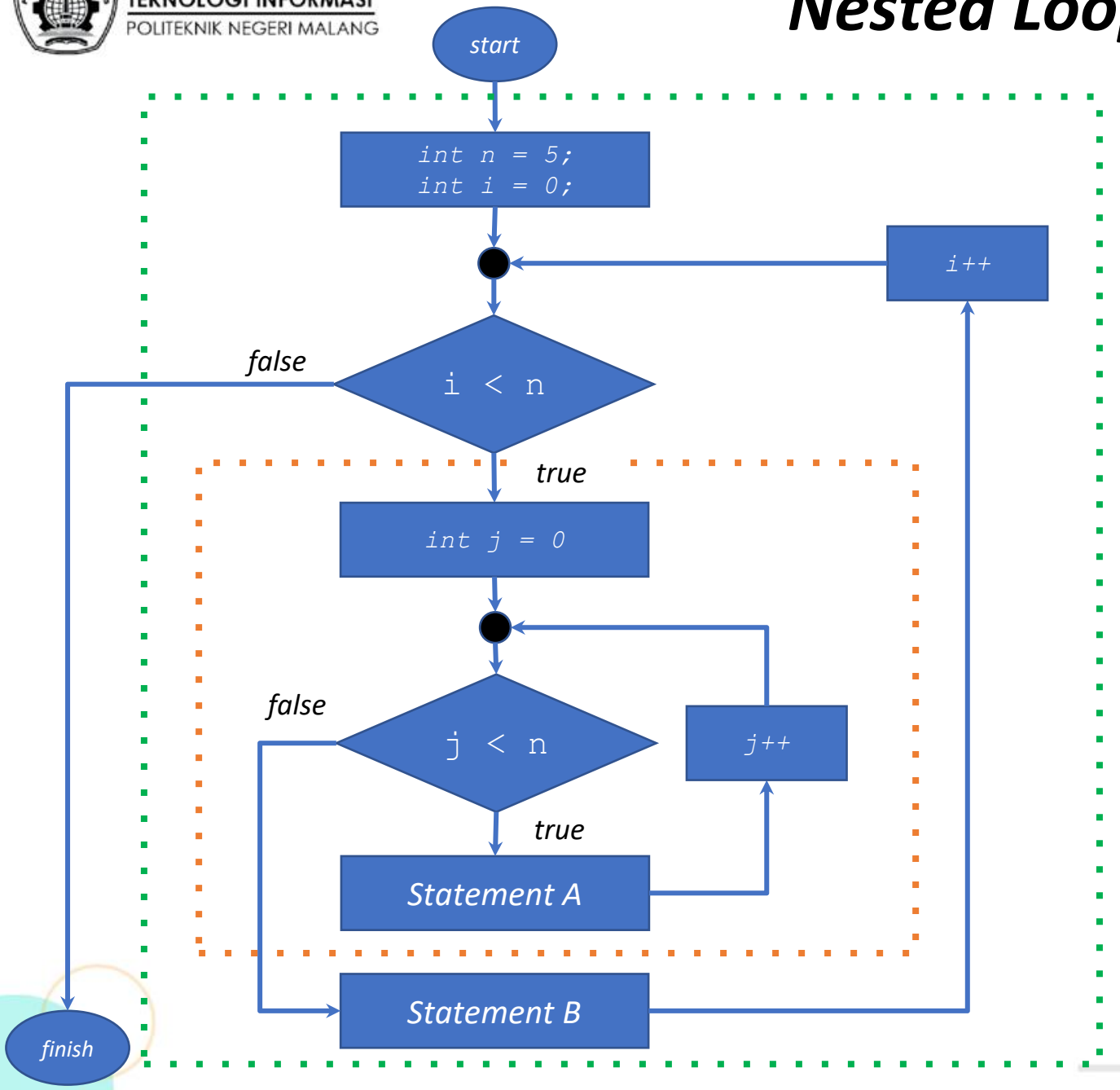
secara umum *flowchart* untuk *nested loop* seperti pada gambar di samping

***Outer loop***

***Inner loop***



# Nested Loop : FOR




```
int n = 5;
for(int i = 0; i < n; i++) {      // loop level 1
    for(int j = 0; j < n; j++) {  // loop level 2
        // statement A
    }

    // statement B
}
```

Outer loop

Inner loop

Outer loop

Inner loop

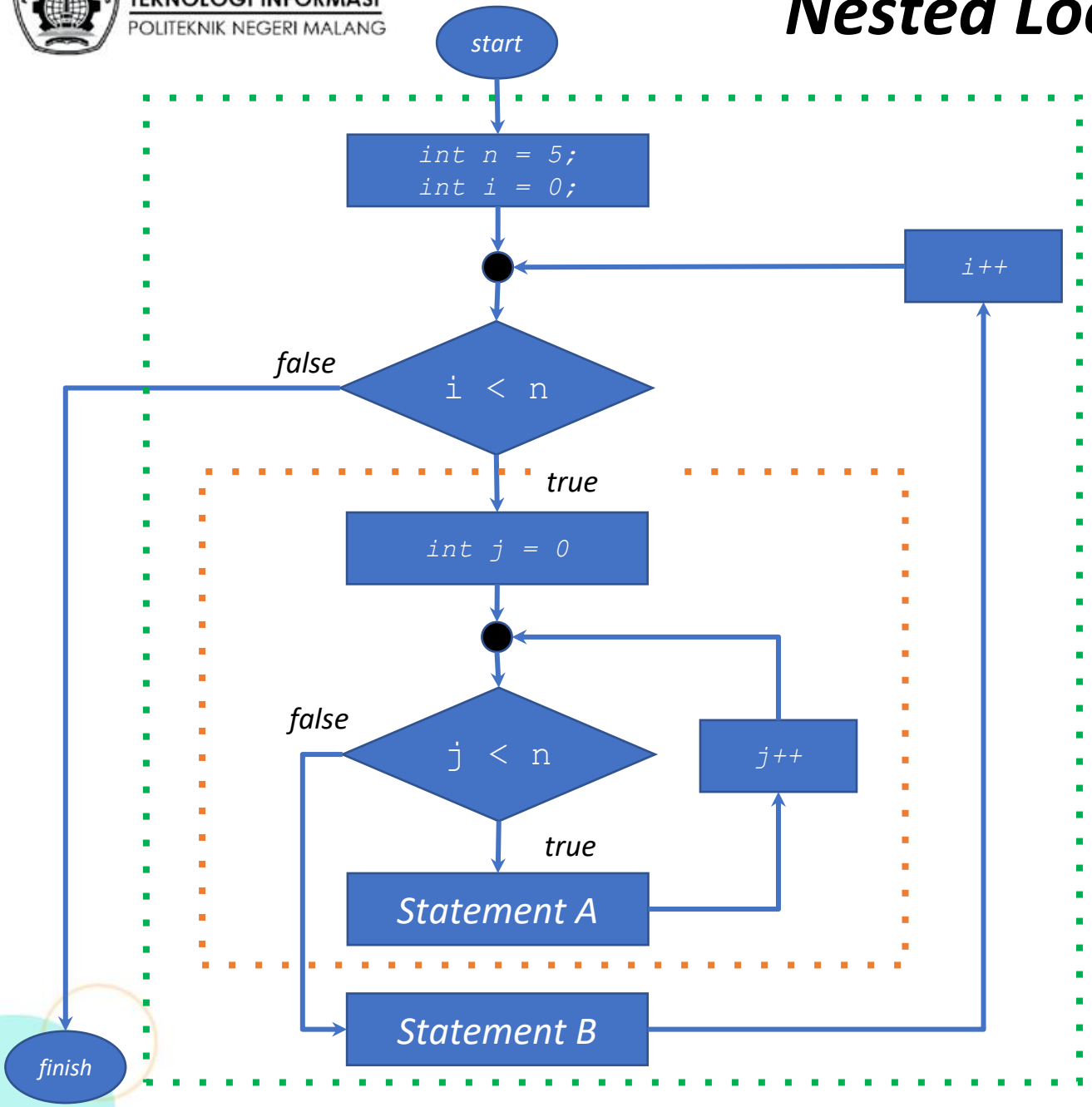
# *Nested Loop : FOR (lebih dari 2 level)*

*Outer loop*

```
4   for(int i = 0; i < n; i++){ //loop level 1
5       for (int j = 0; j < n; j++){    //loop level 2
6           for (int k = 0; k < n; k++){    //loop level 3
7           //statement
8           }
9       }
10      for (int l = 0; l < n; l++){    //loop level 2
11          //statement
12      }
13  }
```

*Inner loop*

# Nested Loop: While




```java
int n = 5;
int i = 0;

// pengecekan loop. Loop terus berjalan selama kondisi (i < n) bernilai true
while(i < n){       // loop level 1
    int j = 0;

    // pengecekan loop. Loop terus berjalan selama kondisi (j < n) bernilai true
    while(j < n){   // loop level 2
        // Statement A
        j++;
    }

    // Statement B
    i++;
}
```

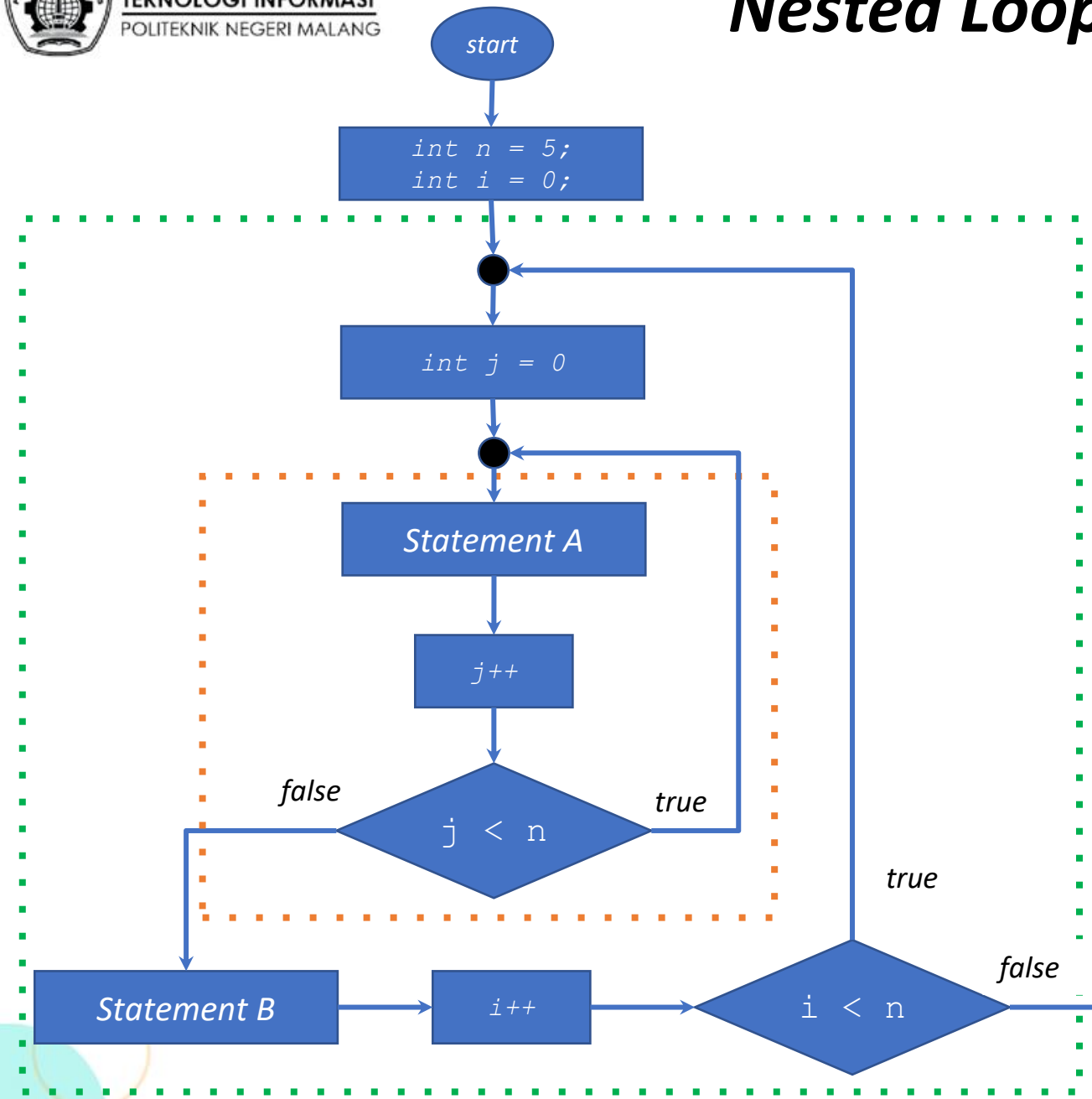
*Outer loop*

*Inner loop*

*Outer loop*

*Inner loop*

# *Nested Loop*: Do-*While*




```
int n = 5;
int i = 0;

do{                 // loop level 1
    int j = 0;

    do{             // loop level 2
        // Statement A
        j++;

    // pengecekan loop. Loop terus berjalan selama kondisi (j < n) bernilai true
    } while(j < n);

    // Statement B
    i++;

// pengecekan loop. Loop terus berjalan selama kondisi (i < n) bernilai true
} while(i < n);
```

*Outer loop*

*Inner loop*

*Outer loop*

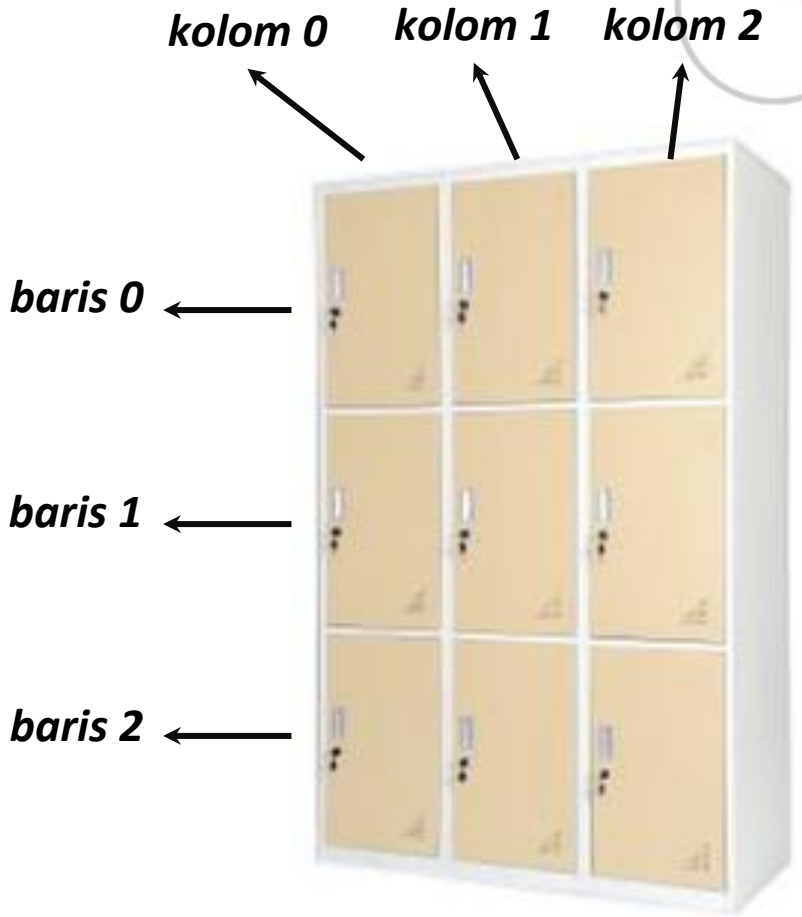
*Inner loop*

# Logika Rak/Loker

***Nested loop*** dengan 2 tingkat/level, ibarat seperti loker.

Dimana ***outer loop*** kita identifikasi sebagai penunjuk **baris** dan ***inner loop*** kita identifikasi sebagai penunjuk **kolom**.

```
for(int baris = 0; baris < 3; baris++) {
    for(int kolom = 0; kolom < 3; kolom++){
        // statement
    }
}
```



# Logika Rak/Loker (cont.)

```
for(int baris = 0; baris < 3; baris++) {
    for(int kolom = 0; kolom < 3; kolom++){
        System.out.print("Baris-"+baris+" & Kolom-"+kolom+". ");
    }
    System.out.println();
}
```



# Kombinasi *Nested Loop*

- *Nested loop* tidak hanya berupa satu jenis *loop*/perulangan yang bertingkat, akan tetapi bisa kombinasi *loop* yang bertingkat.

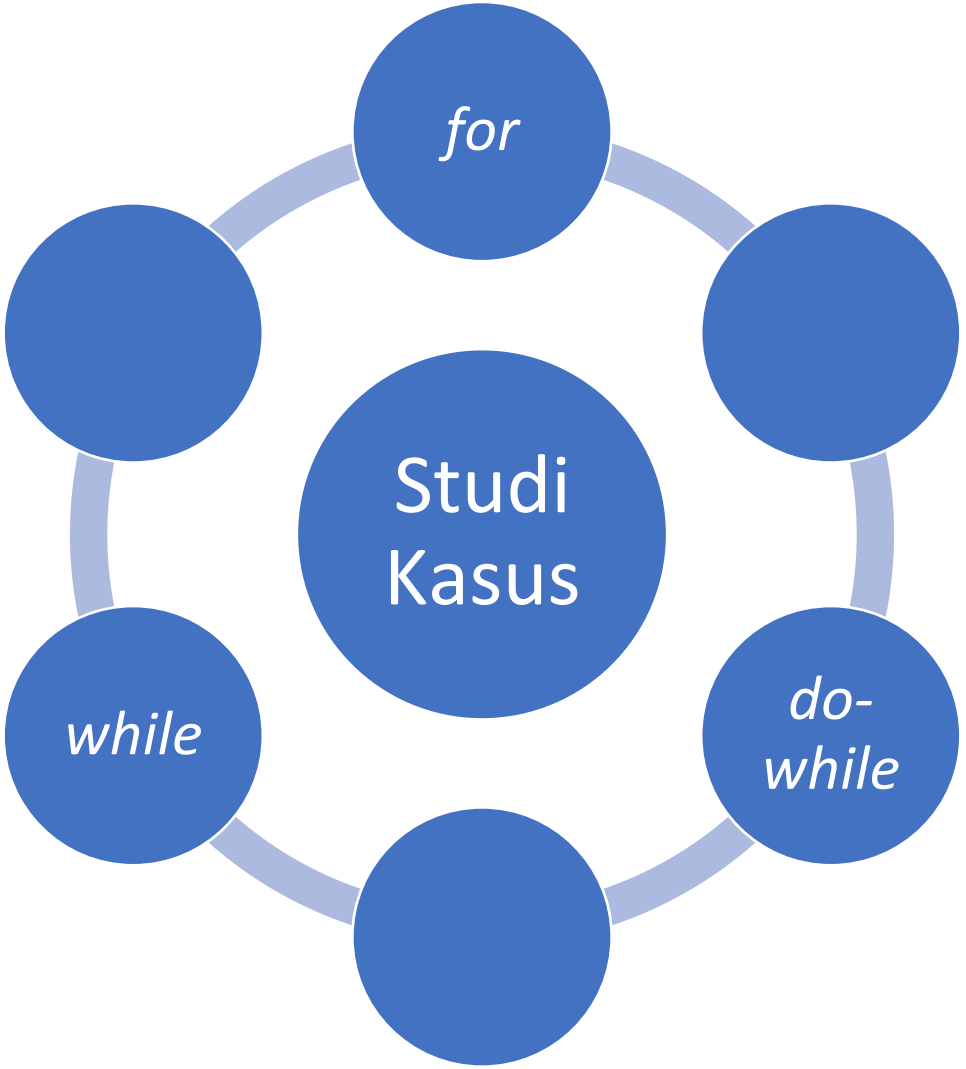
```
/* Kombinasi for dan do-while loop */
for(int i = 0; i < 10; i++){
    int j = 0;
    do {
        // statement
        j++;
    } while(j < 10);
}
```

```
/* Kombinasi while dan for loop */
int i = 0;
while(i < 10) {
    for(int j = 0; j < 10; j++) {
        // statement
    }
    i++;
}
```

```
/* Kombinasi while dan do-while loop */
int i = 0;
while(i < 10) {
    int j = 0;
    do {
        // statement
        j++;
    } while(j < 10);

    i++;
}
```

```
/* Kombinasi do-while dan for loop */
int i = 0;
do {
    for(int j = 0; j < 10; j++) {
        // statement
    }
    i++;
} while(i < 10);
```




Studi Kasus

# Studi Kasus 1 – *Bintang Persegi*



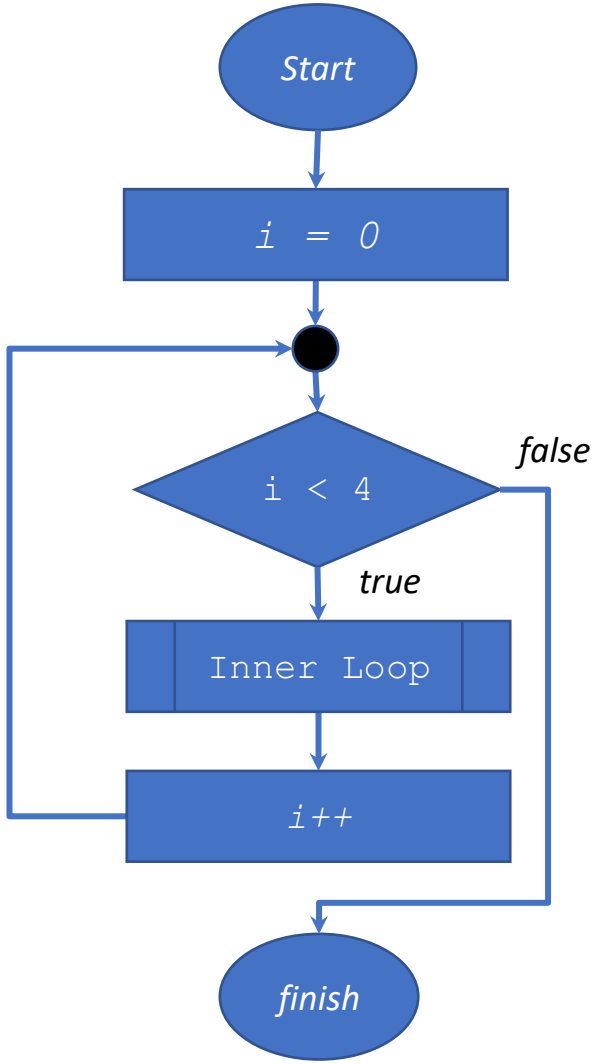

Bagaimana caranya untuk menampilkan tanda * seperti gambar di atas dengan menggunakan nested loop?

Baik menggunakan *nested loop* ***for, while, do-while****?*

# Studi Kasus 1 – Logika Jawaban

Program yang akan dibuat harus ada ***outer loop*** dan ***inner loop***.

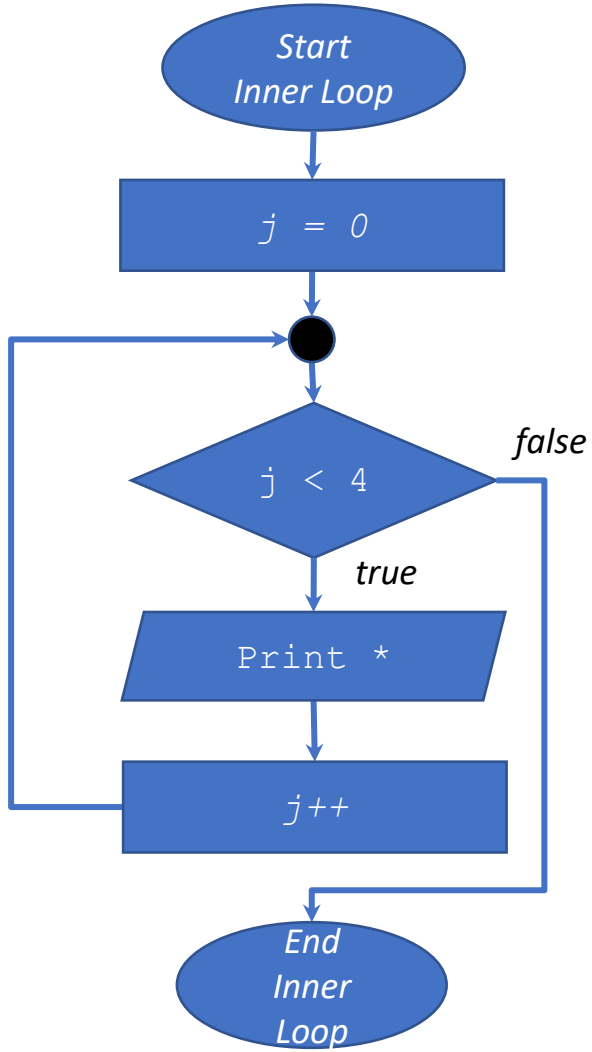
- ***Outer loop*** digunakan untuk menghitung atau melakukan perulangan sebanyak jumlah baris yaitu 4 baris (`i = 0; i < 4; i++`). Setiap *inner loop* selesai di eksekusi, maka akan dibuatkan baris baru.



# Studi Kasus 1 – Logika Jawaban (cont.)

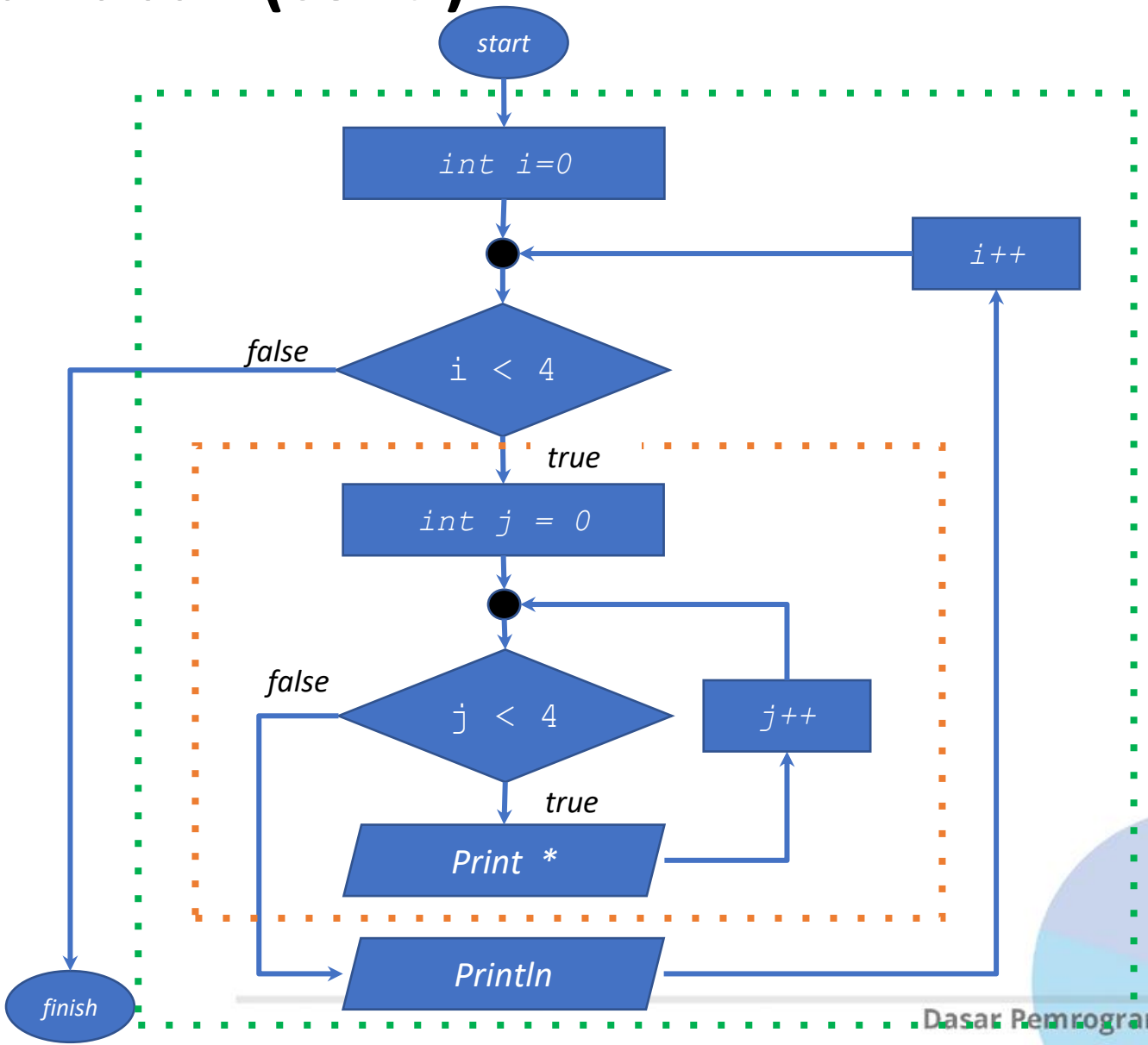
- ***Inner loop*** digunakan untuk mencetak simbol * pada layar, jumlah simbol yang ditampilkan perbarisnya akan menyesuaikan dengan nilai pada variable **j**, yaitu 4 symbol *.

(`j = 0; j < 4; j++`)



# Studi Kasus 1 – Logika Jawaban (cont.)

Gabungkan *flowchart inner* dan *outer* dari *nested loop* studi kasus 1



# Studi Kasus 1 - *FOR*

```
1  public class StudiKasus1 {
2      public static void main(String args[]) {
3          for (int i = 0; i < 4; i++) {
4              for (int j = 0; j < 4; j++) {
5                  System.out.print("*");
6              }
7              System.out.println();
8          }
9      }
10 }
```

*Outer loop* mengulang *output* dari *inner loop*

*Inner loop* menghasilkan satu baris bintang (dengan 4 bintang pada satu baris)

# Studi Kasus 1 - *While*

```
1  public class StudiKasus1 {
2      public static void main(String args[]) {
3          int i = 0;
4          while (i < 4) {
5
6              int j = 0;
7              while (j < 4) {
8                  System.out.print("*");
9                  j++;
10             }
11             System.out.println();
12             i++;
13         }
14     }
15 }
```

*Outer loop* mengulang *output* dari *inner loop*

*Inner loop* menghasilkan satu baris bintang (dengan 4 bintang pada satu baris)

# Studi Kasus 1 - *do-while*

```
1  public class StudiKasus1 {
2      public static void main(String args[]) {
3          int i = 0;
4          do {
5
6              int j = 0;
7              do {
8                  System.out.print("*");
9                  j++;
10             }while (j < 4);
11
12             System.out.println();
13             i++;
14         } while (i < 4);
15     }
16 }
```

*Outer loop* mengulang *output* dari *inner loop*

*Inner loop* menghasilkan satu baris bintang (dengan 4 bintang pada satu baris)

# Studi Kasus 2 – *Bintang Segitiga*

```
*
**
***
****
*****
******
*******
********
*********
**********
```



Bagaimana caranya untuk menampilkan tanda * seperti gambar di atas dengan menggunakan *nested loop*?

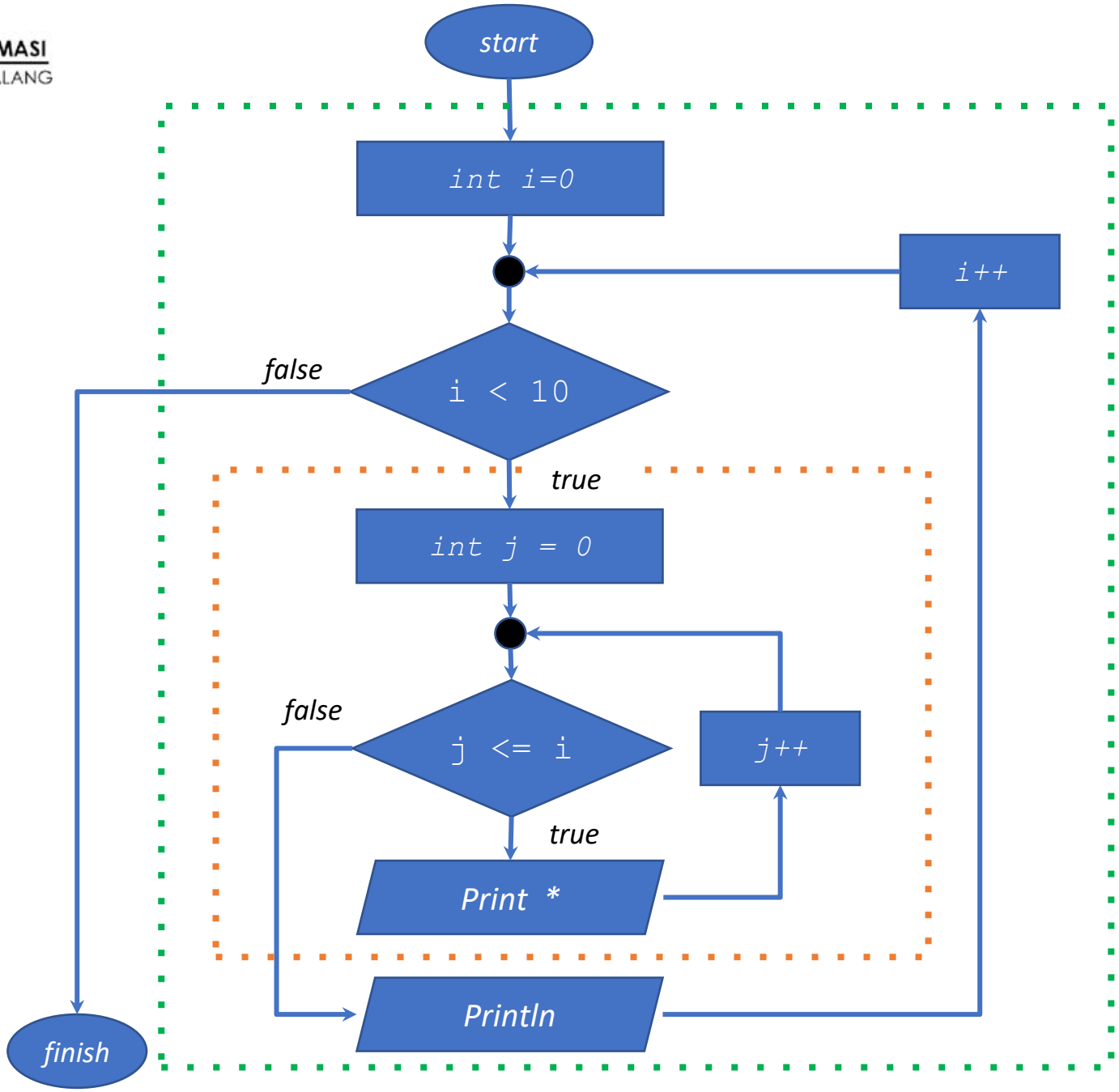
Dimana jumlah bintang dalam satu baris, tergantung nomor baris tersebut (total 10 baris).

# Studi Kasus 2 – Logika Jawaban

Program yang akan dibuat harus ada ***outer loop*** dan ***inner loop***.

- ***Outer loop*** digunakan untuk menghitung atau melakukan perulangan sebanyak jumlah baris yaitu 10 baris (`i = 0; i < 10; i++`). Setiap *inner loop* selesai di eksekusi, maka akan dibuatkan baris baru.
- ***Inner loop*** digunakan untuk mencetak simbol * pada layar, jumlah simbol yang ditampilkan per barisnya akan **menyesuaikan (sama dengan)** dengan nilai/nomor baris pada **baris i** (`j = 0; j <= i; j++`).

- Buatlah *flowchart inner* dan *outer* dari *nested loop* studi kasus 2


start
int i=0
i++
false
i < 10
true
int j = 0
false
j <= i
j++
true
Print *
Println
finish

# Studi Kasus 2 - *FOR*

```
1  public class StudiKasus2 {
2      public static void main(String args[]) {
3          for(int i = 0; i < 10; i++) {
4              for(int j = 0; j <= i; j++){
5                  System.out.print("*");
6              }
7              System.out.println();
8          }
9      }
10 }
```

*Outer loop* mengulang *output* dari *inner loop*

*Inner loop* menghasilkan satu baris bintang yang jumlahnya **menyesuaikan** dengan nilai/nomor baris pada **baris i**

# Studi Kasus 2 - *While*

```
1  public class StudiKasus2 {
2      public static void main(String args[]) {
3          int i = 0;
4          while(i < 10) {
5
6              int j = 0;
7              while(j <= i){
8                  System.out.print("*");
9                  j++;
10             }
11             System.out.println();
12             i++;
13         }
14     }
15 }
```

*Outer loop* mengulang *output* dari *inner loop*

*Inner loop* menghasilkan satu baris bintang yang jumlahnya **menyesuaikan** dengan nilai/nomor baris pada **baris i**

# Studi Kasus 2 – *do-while*

```
1  public class StudiKasus2 {
2      public static void main(String args[]) {
3          int i = 0;
4          do {
5              int j = 0;
6              do {
7                  System.out.print("*");
8                  j++;
9              } while(j <= i);
10
11             System.out.println();
12             i++;
13         } while(i < 10);
14     }
15 }
```

*Outer loop* mengulang *output* dari *inner loop*

*Inner loop* menghasilkan satu baris bintang yang jumlahnya **menyesuaikan** dengan nilai/nomor baris pada **baris i**

# Studi Kasus 3 – Rekap Penjualan Produk Toko

Sebuah toko menjual **3 produk**. Jumlah barang yang terjual dicatat setiap hari selama **4 hari**.

Program diminta untuk:

- membaca jumlah terjual setiap produk per hari melalui keyboard;
- menghitung **total penjualan per produk**;
- menghitung **total penjualan seluruh produk**.

**Contoh data penjualan**

| Produk | H1 | H2 | H3 | H4 | Total |
|---|---|---|---|---|---|
| **Produk 1** | 5 | 3 | 4 | 6 | **?** |
| **Produk 2** | 2 | 4 | 1 | 3 | **?** |
| **Produk 3** | 7 | 5 | 6 | 4 | **?** |
| **Total seluruh produk = ?** | | | | | |

Baris → **produk** (*outer loop*)
Kolom → **hari** (*inner loop*)

# Studi Kasus 3 – Logika Jawaban

Program yang akan dibuat harus ada ***outer loop*** dan ***inner loop***.

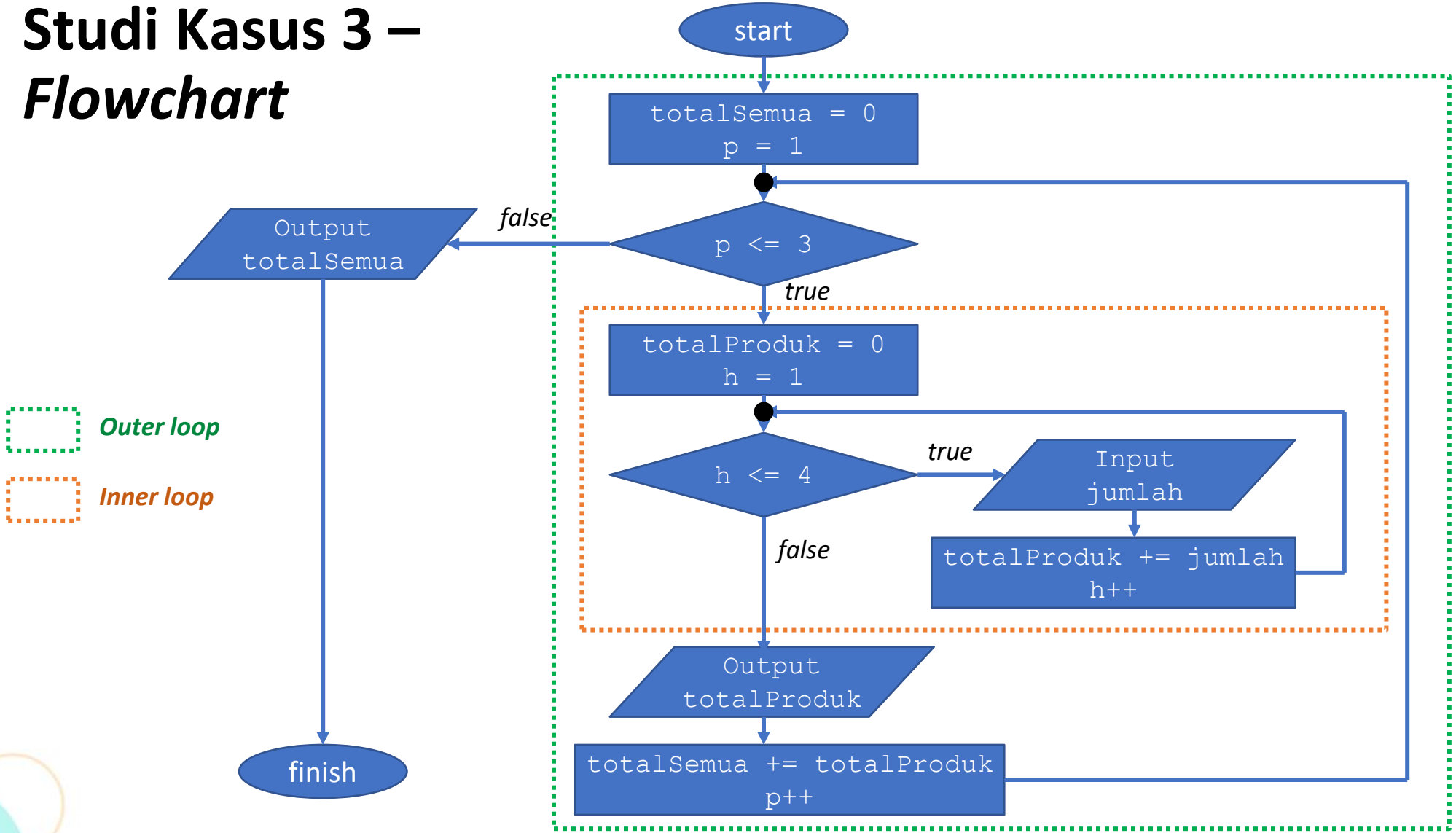
- ***Outer loop*** mengulang proses untuk setiap **produk** (`p = 1; p <= 3; p++`).
- ***Inner loop*** membaca jumlah terjual setiap **hari** (`h = 1; h <= 4; h++`) lalu menambahkannya ke `totalProduk`.
- `totalProduk = 0` ditulis **di dalam** *outer loop* agar di-*reset* setiap kali ganti produk.
- `totalSemua = 0` ditulis **sebelum** *outer loop* karena mengumpulkan total seluruh produk.

**Penelusuran produk 1 (p = 1)**

| h | Jumlah | totalProduk |
|---|---|---|
| - | - | 0 |
| 1 | 5 | 5 |
| 2 | 3 | 8 |
| 3 | 4 | 12 |
| 4 | 6 | **18** |

Cetak total produk 1 = **18**, lalu `totalSemua += 18`.
Saat **p = 2**, `totalProduk` kembali **0**.

# Studi Kasus 3 – *Flowchart*



# Studi Kasus 3 - *FOR*

```
1  import java.util.Scanner;
2
3  public class StudiKasus3 {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          int totalSemua = 0;
7          for (int p = 1; p <= 3; p++) {
8              System.out.println("Produk " + p);
9              int totalProduk = 0;
10             for (int h = 1; h <= 4; h++) {
11                 System.out.print("  Hari " + h + ": ");
12                 int jumlah = sc.nextInt();
13                 totalProduk += jumlah;
14             }
15             System.out.println("Total produk " + p
16                     + " = " + totalProduk);
17             totalSemua += totalProduk;
18         }
19         System.out.println("Total semua produk = " + totalSemua);
20     }
21 }
```

1. `totalSemua` diinisialisasi **sekali**, sebelum *outer loop*.
2. ***Outer loop***: mengulang proses untuk setiap produk (p = 1 s.d. 3).
3. `totalProduk` di-*reset* ke 0 **setiap ganti produk**.
4. ***Inner loop***: membaca penjualan hari 1 s.d. 4 dan menjumlahkannya.
5. Total satu produk ditambahkan ke `totalSemua`.

# Studi Kasus 3 – Contoh Eksekusi dan Output

```
Produk 1
  Hari 1: 5
  Hari 2: 3
  Hari 3: 4
  Hari 4: 6
Total produk 1 = 18
Produk 2
  Hari 1: 2
  Hari 2: 4
  Hari 3: 1
  Hari 4: 3
Total produk 2 = 10
Produk 3
  Hari 1: 7
  Hari 2: 5
  Hari 3: 6
  Hari 4: 4
Total produk 3 = 22
Total semua produk = 50
```

Keterangan: teks kuning = input dari keyboard

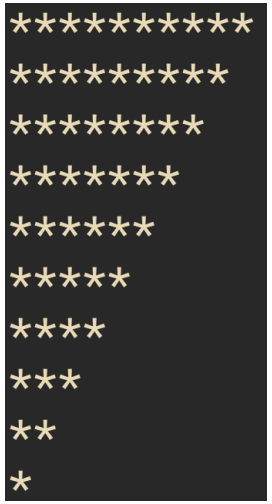
**Jumlah iterasi**

Badan *inner loop* dijalankan **3 × 4 = 12 kali**, sehingga `sc.nextInt()` juga dipanggil 12 kali.

# Latihan

1. Buatlah *flowchart* dari soal gambar di bawah ini

**a**




**b**




2. Buatlah *flowchart* untuk menampilkan deret huruf berikut

```
a
bcd
efghi
jklmnop
qrstuvwxy
```

# Latihan

3. Buatlah *flowchart* dan *pseudocode* untuk menghitung dan menampilkan jumlah kuadrat bilangan 1–n. Gunakan perulangan bersarang. Berikut output yang diharapkan jika n pada rentang 1 s.d 5:

```
n = 1 → jumlah kuadrat = 1
n = 2 → jumlah kuadrat = 1 + 4 = 5
n = 3 → jumlah kuadrat = 1 + 4 + 9 = 14
n = 4 → jumlah kuadrat = 1 + 4 + 9 + 16 = 30
n = 5 → jumlah kuadrat = 1 + 4 + 9 + 16 + 25 = 55
```

# Latihan

4. Buat *flowchart* dan *pseudocode* untuk melakukan rekap penjualan toko dengan ketentuan berikut:

- Jumlah produk dan jumlah hari dimasukkan pengguna (minimal 1).
- Jumlah terjual tidak boleh negatif; jika negatif, input hari tersebut **diulang** (gunakan do-while).
- Tampilkan total dan rata-rata harian tiap produk dan total seluruh produk
- Gunakan while untuk *outer loop* dan for untuk *inner loop*.